

ISTANBUL TECHNICAL UNIVERSITY
Faculty of Computer Science and Informatics

**FLASK-BASED MULTI-DATABASE MANAGEMENT INTERFACE and
WEB-BASED INTERACTIVE FINE-TUNED LLM PLATFORM WITH RAG
INTEGRATION**

INTERNSHIP PROGRAM REPORT

Ilayda Kara

SUMMER / 2025

Istanbul Technical University

Faculty of Computer Science and Informatics

INTERNSHIP REPORT

Academic Year: 2024/2025
Internship Term: ☒ Summer ☐ Spring ☐ Fall

Student Information

Name Surname: İlayda Kara
Student ID: [REDACTED]
Department: Artificial Intelligence and Data Engineering
Program: 100% English
E-Mail: karai22@itu.edu.tr
Mobile Phone: [REDACTED]
Pursuing a Double Major? ☐ Yes ☒ No

In the Graduation Term? ☐ Yes ☒ No

Taking a class at Summer School? ☐ Yes ☒ No

Institution Information

Company Name: İletişim Yazılım
Department: AI and ArGe Department
Web Address: <https://www.iletisimyazilim.com/>
PostalAddress: İhsaniye Mah. İzmir Yolu Cd. Efe Plaza No: 116 Kat: 6 Daire: 11, 16040
[REDACTED]
Nilüfer/Bursa

Authorized Person Information

Department: AI and ArGe Department
Title: ArGe and Project Director
Name Surname: Fatma Hiçyılmaz
Corporate E-Mail: fatma.hicyilmaz@iletisimyazilim.com
Corporate Phone: (0224) 251 41 00

Internship Work Information

Internship Location: ☒ Turkey
☐ Abroad
Internship Start Date: 18.08.2025
Internship End Date: 12.09.2025
Number of Days Worked: 20
During your internship, did you have insurance? ☒ Yes, I was insured by İTÜ.
☐ Yes, I was insured by institution.
☐ No, I did my internship abroad.
☒ No.

Table of Contents

1	INFORMATION ABOUT THE INSTITUTION	1
2	INTRODUCTION	1
3	DESCRIPTION AND ANALYSIS OF THE INTERNSHIP PROJECT	2
3.1	Flask-based Multi-Database Management Interface	2
3.1.1	Technical Architecture and Implementation	2
3.1.2	Database Integration and Synchronization Strategy	3
3.1.3	User Interface Design and User Experience	3
3.2	Web-based Interactive Fine-Tuned LLM Platform with RAG Integration	4
3.2.1	Fine-tuning the Gemma-3 Model	4
3.2.2	RAG System Implementation	6
3.2.3	Adding a Web-based User Interface to Integrated System	8
4	CONCLUSIONS	9
5	REFERENCES	10
6	APPENDIX	12

1 INFORMATION ABOUT THE INSTITUTION

The institution I did my internship is İletişim Bilgisayar Yazılım Sistemleri İhracat İthalat San. ve Tic. A. Ş., commonly known as İletişim Yazılım, located in Nilüfer, Bursa.

Founded in 1994, İletişim Yazılım is a software company, employs around 50 people, with over 50% being engineers. They have an R&D center and a Technopark office in Bursa, and branches or dealers in many cities. The company is certified with ISO standards like quality, information security, customer satisfaction and holds other credentials like patents, “Smart Factory / Smart Enterprise / Smart Hospital” trademarks, and is a Microsoft Gold Partner.

İletişim Yazılım provides technology solutions aimed at enhancing competitiveness of businesses through information systems. Their sectoral coverage includes manufacturing execution systems (MES), business process management (BPM), quality management, digital transformation especially for defense & aerospace, automotive, food, pharmaceuticals, energy, textiles etc. The company develops integration and IoT solutions, as well as demand forecasting and decision support systems tailored for these industries. These projects are designed to provide end-to-end digital transformation, allowing businesses to adapt quickly to changing market dynamics.

One of the institution’s flagship products is CoralReef MES (Manufacturing Execution System), which supports the digitalization of production environments. This system allows factories to collect real-time data from machines, monitor production efficiency, trace products, and manage quality. By integrating with ERP and other enterprise systems, CoralReef MES enables companies to minimize downtime, optimize workflows, and achieve Industry 4.0 standards.

Another significant solution is Manage Mind BPM (Business Process Management), which provides organizations with the ability to design, automate, and monitor their business processes. Through customizable workflows, form-based applications, and performance dashboards, ManageMind BPM helps companies improve operational transparency, increase employee productivity, and ensure compliance with regulations.

Methodologically, they focus on processes around: gathering data from machines or operations, integrating various data sources (ERP, MES, others), dashboards and visualizations, KPI / performance monitoring, user-defined forms and reporting, continuous improvement, and quality / security standards. Also, there is an internal academy (İletişim Akademi) that supports training and knowledge dissemination.

During my internship, I worked on two major projects:

- Flask-based Multi-Database Management Interface
- Web-based Interactive Fine-Tuned LLM Platform with RAG Integration

These projects are aligned with the institution’s methods of integrating multiple data sources, building tools for oversight, and utilizing advanced machine learning / large language models (LLMs) to enhance decision support and automate processes. My work contributes by extending the company’s capability in AI/ML tools, helping build interfaces and platforms that can be deployed within clients’ digital transformation initiatives.

In short, my contributions support the company’s goals of providing modern, secure, data-driven solutions, improving responsiveness, flexibility, and adding AI capabilities to its product offering.

2 INTRODUCTION

The purpose of this internship was to gain practical experience in software development by working on two projects that combine database management and artificial intelligence. The overall scope was to strengthen technical skills in web technologies, machine learning, and system integration, while contributing to the institution’s vision of digital transformation.

- **Flask-based Multi-Database Management Interface:** The first project, Flask-based Multi-Database Management Interface, aimed to create a web application for managing different database systems in a single environment. The platform supported both Microsoft SQL Server (MSSQL) and MongoDB, allowing users to browse tables, view records, and perform operations through a simple Flask-based interface. This project showed how relational and non-relational databases can be unified under one system, which is important for organizations that work with diverse data sources.
- **Web-based Interactive Fine-Tuned LLM Platform with RAG Integration:** The second project, Web-based Interactive Fine-Tuned LLM Platform with RAG Integration, had a broader and more research-oriented scope.
 - **Doing research and studying LLM's:** Since I haven't taken NLP course yet, I began with a study of large language models (LLMs) and related subjects, including the structure, fine-tuning methods, and use cases of LLM's, concepts like tokenization, embedding, LoRA, zero-few shot learning, RAG and their structure and use cases etc.
 - **LLM Gemma Fine-Tuning and Rag System Integration:** I performed fine tuning on the Gemma 3-4B model using the Unsloth library with 4-bit quantization and supported it with a small instruction-response dataset. This improved the model's ability to generate more context-aware answers. Then I integrated the fine-tuned model into a Retrieval-Augmented Generation (RAG) pipeline, using ChromaDB for vector storage and the Multilingual-e5 embedding model for semantic search. This integration allowed the system to retrieve relevant documents for each query and use them as context during response generation.
 - **Adding a Web-based User Interface to Integrated System:** Finally, I designed a user-friendly web interface using Gradio, for a chatbot-like appearance. Through this interface, users could interact with the fine-tuned model, test retrieval results, and experience an end-to-end LLM-based application.

3 DESCRIPTION AND ANALYSIS OF THE INTERNSHIP PROJECT

In this section I will be explaining these projects I completed during my internship; a flask-based multi-database management interface and a web-based interactive fine-tuned LLM platform with RAG integration thoroughly and giving the necessary details.

3.1 Flask-based Multi-Database Management Interface

3.1.1 Technical Architecture and Implementation

I developed the Flask-based Multi-Database Management Interface as a unified platform for managing heterogeneous database systems within a single web application. The system architecture is built upon Flask, following the Model-View-Controller (MVC) paradigm to separate concerns and facilitate scalability. For database integration, I employed PyODBC with ODBC Driver 17 to establish reliable connectivity to Microsoft SQL Server (MSSQL), while PyMongo was used to interface with MongoDB.

To address different operational requirements, I implemented the project in two distinct modes. In the first mode, MSSQL functions as the primary data source, while MongoDB acts as a synchronized

replica. Here, I designed a real-time synchronization mechanism that propagates every Create, Read, Update, and Delete (CRUD) operation performed on MSSQL directly into MongoDB. This bidirectional synchronization ensures data consistency across relational and document-based environments, enabling seamless interoperability.

In the second mode, I designed the system to allow independent database operations, where changes in one database do not propagate to the other. In this configuration, MSSQL and MongoDB operate as fully autonomous systems, enabling separate experimental workflows and eliminating dependency between relational and document-based storage. Although the overall system structure and interface remain identical in both implementations, the synchronization logic is disabled in the independent mode, allowing flexibility in testing different database management strategies.

3.1.2 Database Integration and Synchronization Strategy

In the synchronized configuration, I implemented a transactional synchronization pipeline that ensures robust data integrity. Each modification applied to MSSQL—whether insertion, update, or deletion—triggers a corresponding mirrored operation in MongoDB. During initialization, the system first retrieves records from the MSSQL ‘Persons’ table, clears existing entries in the MongoDB collection, and inserts a consistent dataset. This guarantees structural alignment between the two databases before real-time synchronization begins.

For updates, I utilized MongoDB’s upsert functionality, which intelligently handles the insertion of new documents if corresponding entries do not already exist, thereby preventing data divergence. Deletion operations are executed in parallel across both systems, ensuring that removal in one database is accurately reflected in the other. This design allows MSSQL to preserve its relational integrity while MongoDB capitalizes on its flexible schema and scalability.

In contrast, the independent configuration eliminates these synchronization triggers entirely. Here, operations on MSSQL remain isolated from MongoDB, and vice versa. This alternative design supports use cases where decoupled database evolution is preferable, such as testing schema modifications in MongoDB without impacting the relational structure of MSSQL, or benchmarking system performance under varying database workloads.

For deployment flexibility, I integrated environment-based configuration management using ‘.env’ files. This allows secure storage of credentials and dynamic adaptation to different server environments, thereby supporting development, testing, and production contexts without code-level modifications.

3.1.3 User Interface Design and User Experience

The user-facing interface was implemented using Bootstrap 5.3.3, ensuring a responsive and modern layout consistent with professional web application standards. I designed the dashboard with a clean, card-based architecture, leveraging subtle shadow effects and refined typography for readability. Data is presented in striped tables with hover states to enhance visual clarity during navigation.

Interactive elements follow semantic color conventions: primary blue for record creation, warning orange for edit operations, and danger red for deletions. Form interfaces enforce validation protocols to prevent incorrect submissions, while error messages and success indicators provide immediate user feedback. This ensures not only usability but also data integrity, since invalid inputs are systematically filtered before being processed by either database.

3.2 Web-based Interactive Fine-Tuned LLM Platform with RAG Integration

This part focused on developing a comprehensive web-based interactive fine-tuned Large Language Model platform integrated with Retrieval-Augmented Generation capabilities. My primary objective was to create an end-to-end system that combines the language model fine-tuning techniques with efficient information retrieval mechanisms, and lastly delivering an accessible web interface for users to interact with the enhanced model.

The project was structured around three main technical components:

- Fine-tuning the Gemma-3 model using advanced optimization techniques,
- Implementing a robust RAG system with vector database integration,
- Developing a user-friendly web interface to make the system accessible to end users.

Throughout the development process, my focus was placed on memory efficiency, performance optimization, and systematic evaluation of model capabilities.

3.2.1 Fine-tuning the Gemma-3 Model

3.2.1.1 Model Architecture and Configuration

I began with the model Gemma-3 using the Unsloth library to enable efficient training with limited computational resources. To achieve that I created a token in HuggingFace and load the model using this token. The model was loaded in its 4-bit quantized version, which really reduces the memory usage in GPU memory requirements without significantly sacrificing performance and also provides faster gradient computation and reduced I/O overhead. The instruction-following capability of the IT variant demonstrated a good performance in understanding and executing complex user instructions. And the maximum sequence length was set to 2048 tokens, since it makes sure that the model could handle extended user queries and responses.

I also utilized Low-Rank Adaptation (LoRA) to fine-tune specific components of the Gemma-3 transformer architecture. Parameters such as query, key, value, output, gate, up, and down projection layers were optimized, with LoRA. The hyperparameters can be seen in Table 1 [1]. This configuration provided a balance between model adaptability and regularization, which reduces overfitting risks while maintaining performance. Additionally, GPU memory usage was continuously monitored and optimized to provide efficient allocation of resources during the training process.

3.2.1.2 Dataset Preparation and Augmentation

I prepared the dataset in instruction-response format, which is widely adopted for chat-based fine-tuning tasks. This original dataset consists of 200 examples covering topics such as deep learning, large language models (LLMs), machine learning, computer vision, data mining, and databases, along with the associated conceptual knowledge, since I fine-tune this model to achieve domain-specific adaptation. To improve the diversity and generalization of the model, I applied data augmentation techniques. Using a predefined set of linguistic transformation patterns, the dataset was expanded by rephrasing existing instructions into multiple variations. For instance, questions like “What is X?” were reformulated as “Can you explain X?” or “Tell me more about X.”

Through this augmentation, the dataset was expanded to 575 examples, achieving approximately a $2.9\times$ increase in size. The final split allocated 402 samples for training, 86 for validation, and 87 for testing, following a 70/15/15 distribution. The responses were analyzed for word length; short (<15), medium (15-25), and long (>25) categories, which supported balanced training. After that, the instruction-response pairs were converted into the Gemma prompt format. A Python function (format_prompts) applied the template can be seen in the Code Listing 1.

Table 1: Training Hyperparameters for Fine-Tuning the Gemma-3 Model

HyperParameter&Parameter	Value
LoRA Rank (r)	16
Alpha (lora_alpha)	16
LoRA Dropout	0.1
Optimizer	AdamW(8bit)
Physical Batch Size (processed by GPU)	1
Gradient Accumulation Steps	8
Effective Batch Size	8 (1 × 8)
Max steps (epochs)	200
Warmup Steps	10
Weight Decay	0.01
LR Scheduler Type	linear
Learning Rate	2e-4

```

1 <bos><start_of_turn>user
2 {instruction}<end_of_turn>
3 <start_of_turn>model
4 {response}<end_of_turn><eos>
5

```

Listing 1: Gemma prompt format

This ensured alignment with the model’s expected conversational structure. Training, validation, and test datasets were converted into pandas DataFrames and subsequently into HuggingFace Dataset objects. Each dataset was mapped using the formatting function to ensure consistent data representation.

3.2.1.3 Training Implementation and Optimization

The fine-tuning was conducted using the SFTTrainer class, which handled the training loop, evaluation, and logging of metrics. A custom TrainingMetricsCallback was developed to monitor training loss, validation loss, and learning rates while supporting early stopping.

During training, I logged losses every two steps and conducted evaluations every ten steps. I applied an early stopping mechanism in which the validation loss was monitored with a patience of five evaluations and a minimum delta of 0.001, meaning that training was terminated early if no improvement was observed across five consecutive evaluations. To regulate optimization, I implemented a linear learning rate decay schedule (lr_scheduler_type=”linear”) with an initial warmup phase of 10 steps, and I visualized the learning rate dynamics throughout training. These visualizations provided confirmation of the scheduling effect and offered insights into how the warmup stabilized the initial optimization phase.

I incorporated real-time debugging capabilities that enabled immediate detection of instabilities, loss explosions, or convergence issues during training. I continuously monitored GPU memory usage to prevent memory overflow errors and implemented a comprehensive error-handling framework to ensure the reliability of the metric system. Despite occasional system interruptions, data collection and subsequent analysis were not compromised. Furthermore, I facilitated troubleshooting by explicitly reporting missing data types, which streamlined the resolution of potential issues.

The training process achieved convergence with the following performance metrics:

- Final training loss: 0.25
- Final validation loss: 0.42

For the training and validation loss curve, see Figure 2. This relatively low training loss indicates that the model learned to minimize prediction errors on the training dataset. In the context of language modeling, this value suggests relatively strong pattern recognition within the training data, effective parameter adaptation through LoRA fine-tuning and successful convergence without apparent underfitting. But of course this can be improved more with a bigger dataset. Because, the validation loss being higher than training loss reveals important characteristics of the model's generalization capability such as:

- **Generalization Gap:** The difference ($0.425214 - 0.259400 = 0.165814$) indicates moderate overfitting, which is typical for fine-tuned language models on specialized datasets. But,
- **Acceptable Overfitting Range:** This gap represents approximately 64% increase over training loss, which falls within acceptable bounds for small dataset fine-tuning scenarios. Also,
- **Model Stability:** The validation loss remains below 0.5, suggesting the model maintains reasonable performance on unseen data.

As I mentioned, these results are largely influenced by the use of a limited dataset that I manually created. Since the data was further divided into training, validation, and test sets, the amount available for training became relatively small. Also while systematic, the augmentation focused primarily on question reformulation rather than response diversification, which may not allow the model's exposure to more varied reasoning patterns. Implementing response-level augmentation (paraphrasing, style variation) in addition to question reformulation would have been more beneficial. Also, including negative examples and edge cases to improve robustness might have been contributed too. Nevertheless, the model demonstrated stable convergence and promising performance, which suggests that even stronger results could be achieved with a larger dataset.

3.2.2 RAG System Implementation

After fine-tuning, I designed and implemented a RAG system to extend the model's knowledge base by integrating external document retrieval with contextual response generation. For efficient similarity search and seamless integration with the Python ecosystem, I adopted ChromaDB as the vector database. To generate semantic representations, I employed the `intfloat/multilingual-e5-small` embedding model, selected for its multilingual coverage, compact architecture for memory efficiency, and strong performance on semantic similarity benchmarks. Knowledge base documents were systematically segmented into coherent chunks, transformed into dense vector embeddings, and stored in ChromaDB with relevant metadata. To support scalable retrieval, I applied HNSW indexing, enabling high-recall similarity search across large collections.

During query processing, user inputs were embedded using the same E5 model with the "query:" prefix to ensure alignment with document embeddings. ChromaDB then performed cosine similarity search to retrieve the top-k most relevant documents, which I concatenated into a coherent contextual background. This context was incorporated into Gemma's instruction format through prompt engineering, enabling the fine-tuned model to generate responses grounded in retrieved evidence. To evaluate system performance, I conducted experiments with five comprehensive test queries, measuring context relevance, answer quality, response length consistency, and retrieval success rate. The results demonstrated

that the proposed RAG pipeline effectively enhanced contextual accuracy, improved robustness, and increased the reliability of generated responses.

The following diagram in Figure 1 illustrates the step-by-step operation of the RAG pipeline implemented in the `rag_query()` function, detailing how a user query is transformed into a contextually grounded response through query preprocessing, semantic embedding, vector similarity search, document retrieval, prompt engineering, model inference, and post-processing.

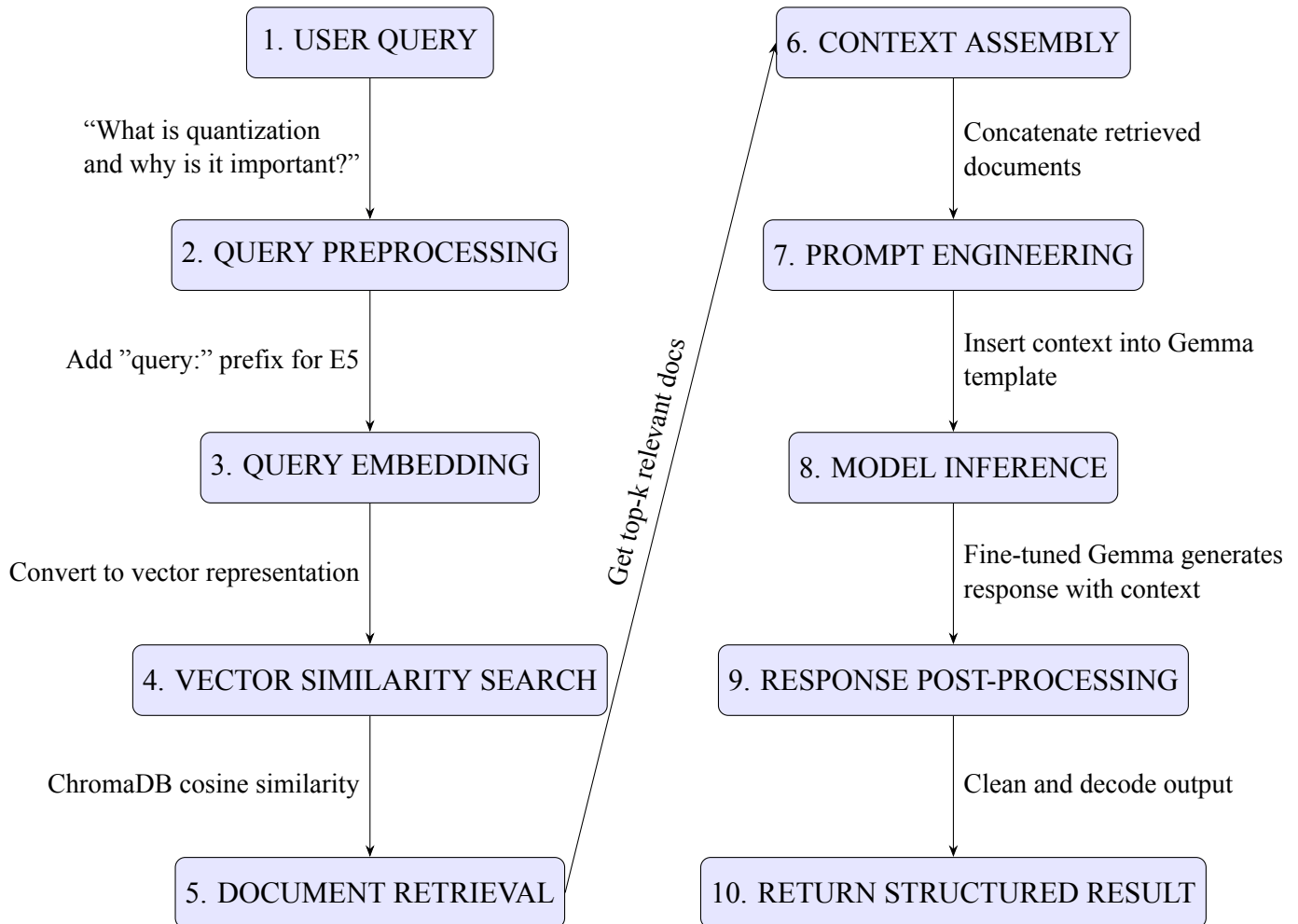


Figure 1: RAG Pipeline

I conducted a comprehensive evaluation of the RAG system across four key performance dimensions as in Figure 5, which provided critical insights into its strengths and limitations.

- Semantic Similarity : 0.901
- Retrieval Quality : 0.787
- Answer Completeness : 0.684
- Context Utilization : 0.508
- Overall Score : 0.704

The system achieved an excellent semantic similarity score of 0.901, demonstrating that the multilingual E5 embedding model effectively captured semantic meaning and that ChromaDB's cosine similarity search yielded accurate query–document alignment with minimal false positives. Retrieval quality was also strong, with a score of 0.787, reflecting the effectiveness of top-k selection and

HNSW indexing in ranking relevant documents from the knowledge base. The overall system score of 0.704 indicated balanced performance across components, validating the integration of retrieval and generation processes while still leaving room for optimization. Answer completeness received a moderate score of 0.684, suggesting that generated responses can have more depth and essential details. Context utilization, with a score of 0.508, shows that the model should leverage retrieved evidence more consistently during response generation.

Taken together, these findings suggest that while the system demonstrates excellent semantic matching and robust retrieval quality which indicates reliable performance suitable for deployment, though its effectiveness could be further enhanced through targeted improvements. In particular, context utilization score points to the need for more effective prompt engineering and fine-tuning strategies that encourage the integration of external evidence. Additionally, supplementing the knowledge base with more detailed and interconnected documents could improve both completeness and contextual grounding.

3.2.3 Adding a Web-based User Interface to Integrated System

3.2.3.1 System Functions and Core Operations for Interface

The query processing function is designed as the central component for handling user interactions with the system. It accepts key input parameters including the user query ('user_question'), the creativity parameter ('temperature'), and the maximum response length ('max_tokens').

The execution pipeline consists of three sequential stages. First, input validation mechanisms detect empty or invalid queries. Second, the response generation process is executed. Finally, metadata such as processing time is appended for transparency before delivering the response to the user.

For real-time monitoring, `get_system_status` function continuously evaluates the operational state of the system. This function tracks the loading status of the language model, the availability and size of the RAG document store, the health of the embedding model, GPU memory consumption, and overall device utilization. By consolidating these metrics, the system provides immediate diagnostic insights that support debugging and performance checks.

3.2.3.2 Interface Design

I used Gradio for web interface to provide an accessible and user-friendly interaction layer as seen in Figure 8. The main Q&A section consists of a multi-line text input for user queries, action buttons for submission and clearing, and a formatted display area for generated responses. An advanced settings panel enables users to toggle the RAG system, adjust creativity using a temperature slider (0.1–1.0), and configure token limits between 50–512 tokens. Additionally, a monitoring panel provides real-time system status and health check functions.

To enhance usability, the interface includes a set of predefined example queries covering topics such as machine learning fundamentals, advantages of RAG systems, distinctions between fine-tuning and transfer learning, ChromaDB features, and embedding model principles.

The interface was refined with custom CSS, including a gradient header design and styled status boxes, ensuring a professional and accessible user experience.

3.2.3.3 Interface Initialization and Control

A structured initialization sequence ensures stable deployment. The process includes verification of system readiness, execution of simple health checks, instantiation of the Gradio interface, and the generation of a publicly accessible URL when executed. Configurable parameters include debug mode activation, error display, server port binding (default 7860), and universal IP accessibility (0.0.0.0).

Through the integration of the Gemma 3 model with the RAG system into a Gradio-based web interface, a functional and user-friendly platform was established. The combination of rapid interface prototyping, basic monitoring functions, and useful design significantly enhances the system's usability.

4 CONCLUSIONS

In this section, I evaluate the work completed during my internship at İletişim Yazılım. This internship provided me a valuable opportunity to translate theoretical knowledge into practical applications across two interdisciplinary projects: the Flask-based Multi-Database Management Interface and the Web-based Interactive Fine-Tuned LLM Platform with RAG Integration. Both projects combined aspects of software engineering, database management, and artificial intelligence, thereby allowing me a comprehensive understanding of how modern systems are designed, implemented, and deployed in real-world contexts.

The first project underlined the integration of heterogeneous databases into a unified management platform. By developing a Flask-based interface that supported both Microsoft SQL Server (MSSQL) and MongoDB, the project demonstrated the useful aspect of unifying relational and non-relational databases under a single operational framework. The implementation of two modes, synchronized and independent, showed how organizations can choose between strict data consistency and flexible autonomy depending on their use cases. Technical challenges, such as real-time synchronization, transactional integrity, and bidirectional updates, were resolved through the application of robust programming practices and careful architecture design. Additionally, the user interface design showed the importance of usability and accessibility in database applications.

The second project extended into more advanced and exploratory domains by combining large language models (LLMs) with retrieval-augmented generation (RAG). The fine-tuning of the Gemma model with Low-Rank Adaptation (LoRA), supported by instruction-response datasets and augmentation strategies, highlighted the process of adapting general purpose models to specialized domains. While dataset limitations introduced certain challenges, the training results demonstrated the potential of even small-scale fine-tuning to achieve meaningful improvements in contextual accuracy. The subsequent integration of the fine-tuned model into a RAG pipeline further showed how external knowledge sources can complement model capabilities, enhancing contextual grounding and retrieval quality. The evaluation metrics revealed both strengths, such as high semantic similarity and reliable retrieval, and weaknesses, particularly in context utilization and answer completeness. These results provided a balanced perspective on the current state of the system and offered clear directions for further improvement, such as expanding datasets, enhancing prompt engineering, and varying augmentation strategies.

Beyond technical accomplishments, the internship also provided me essential skills in project planning, systematic experimentation, and problem-solving. In particular, debugging GPU memory issues, designing synchronization pipelines, and building interactive web interfaces required adaptability. Furthermore, the projects required continuous learning and independent research into areas such as database synchronization strategies, quantization techniques, LoRA optimization, and semantic embedding models.

The integration of system-level engineering with user-facing design represented another key achievement. In both projects, technical functionality was matched by accessible and professional interfaces—Bootstrap-based dashboards for the database system and a Gradio-powered chatbot interface for the LLM platform. These designs supported the principle that technology must be not only efficient and scalable but also usable and approachable for end users.

In summary, the internship succeeded in its primary aim of bridging academic knowledge with applied expertise. It offered exposure to both industry-standard tools and cutting-edge research directions, creating a balanced experience that combined practicality with innovation. The Flask-based database management project strengthened competence in web frameworks, data synchronization, and system integration, while the fine-tuned LLM with RAG integration advanced skills in machine learning, natural language processing, and vector-based retrieval systems. Together, these projects contributed to a comprehensive learning both technical proficiency and professional development.

Here, in Table 2 is a summarized table of the tasks I have completed

Table 2: Summary Table of Tasks

Task Name	Task Overview	Skills/Knowledge Gained
Flask-based Multi-Database Management Interface	Developed a web platform to manage MSSQL and MongoDB, with modes for synchronized and independent operation.	Flask web development, database integration (PyODBC, PyMongo), real-time synchronization, MVC design, environment configuration, UI/UX design with Bootstrap.
Database Synchronization Strategy	Designed real-time CRUD synchronization between MSSQL and MongoDB	Data consistency, bidirectional synchronization, schema alignment, operational flexibility, transactional design.
Dataset Preparation and Augmentation	Expanded dataset through linguistic transformation and formatted instruction-response pairs for model training.	Data augmentation, dataset structuring, prompt engineering, evaluation splitting, systematic preprocessing.
LLM Fine-Tuning (Gemma-3 with LoRA)	Fine-tuned a quantized Gemma model using LoRA with learning rate scheduling, gradient accumulation, and early stopping	NLP fundamentals, LoRA optimization, quantization, loss analysis, overfitting management, GPU resource management, error handling, debugging strategies
RAG System Implementation	Integrated fine-tuned model with ChromaDB and multilingual-E5 embeddings for retrieval-augmented generation.	Vector embeddings, semantic search, HNSW indexing, context integration, system evaluation.
Web Interface for LLM + RAG	Built an interactive Gradio interface with query input, response display, settings panel, and real-time monitoring.	Gradio interface design, front-end/backend integration.

5 REFERENCES

- [1] Talebi, S. (2023, September 11). Fine-Tuning Large Language Models (LLMs): A conceptual overview with example Python code. Towards Data Science. <https://towardsdatascience.com/fine-tuning-large-language-models-llms-23473d763b91/>
- [2] Stöffelbauer, A. (2023, October 24). How Large Language Models Work: From zero to ChatGPT. Data Science at Microsoft. <https://medium.com/data-science-at-microsoft/how-large-language-models-work-91c362f5b78f>
- [3] Bento, C. (2025, January 21). Large Language Models: A Short Introduction. TDS Archive. <https://medium.com/data-science/large-language-models-a-short-introduction-bb8366118ad0>
- [4] Kesrwani, A. (n.d.). What are LLM (Large Language Model)? <https://medium.com/@akash.kesrwani99/what-are-llm-large-language-model-51d1315acaf4>

- [5] Naveed, H., Khan, A. U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., & Mian, A. (2023). A comprehensive overview of large language models. arXiv. <https://arxiv.org/pdf/2307.06435>
- [6] Large Language Models (LLMs) (n.d.). IBM. IBM Think.<https://www.ibm.com/think/topics/large-language-models>
- [7] Large Language Model (LLM) tutorial. GeeksforGeeks. (n.d.).<https://www.geeksforgeeks.org/deep-learning/large-language-model-llm-tutorial/>
- [8] Fine-tuning Large Language Models (LLMs). GeeksforGeeks. (n.d.).<https://www.geeksforgeeks.org/deep-learning/fine-tuning-large-language-model-llm/>
- [9] Exploring the technical architecture behind Large Language Models. GeeksforGeeks. (n.d.). <https://www.geeksforgeeks.org/artificial-intelligence/exploring-the-technical-architecture-behind-large-language-models/>
- [10] RAG (Retrieval-Augmented Generation) architecture. GeeksforGeeks. (n.d.).<https://www.geeksforgeeks.org/nlp/rag-architecture/>
- [11] Abhyuday, T. (2023, August 9). Retrieval-Augmented Generation (RAG): From basics to advanced. Medium.<https://medium.com/@tejpal.abhyuday/retrieval-augmented-generation-rag-from-basics-to-advanced-a2b068fd576c>
- [12] Moraneus. (2024, February 4). Python Flask: A comprehensive guide from basic to advanced. Medium. <https://medium.com/@moraneus/python-flask-a-comprehensive-guide-from-basic-to-advanced-fbc6ec9aa5f7>
- [13] Flask tutorial. GeeksforGeeks. (n.d.).<https://www.geeksforgeeks.org/python/flask-tutorial/>
- [14] Chakole, A. (2022, December 18). Migrating data from SQL Server to MongoDB. Medium.<https://medium.com/@anupchakole/migrating-data-from-sql-server-to-mongodb-8bdfb77dff6b>
- [15] Kago, A. (2025). Why and when MongoDB is faster than MySQL: A mathematical analysis. Medium.<https://medium.com/@a.kago1988/why-and-when-mongodb-is-faster-than-mysql-a-mathematical-analysis-2373ab9f6936/>
- [16] MongoDB vs MySQL. GeeksforGeeks. (2025, July 14).<https://www.geeksforgeeks.org/mongodb/mongodb-vs-mysql/>
- [17] Creating user interfaces for AI models using Gradio. GeeksforGeeks. (n.d.).<https://www.geeksforgeeks.org/artificial-intelligence/python-creating-user-interfaces-for-ai-models-using-gradio/>
- [18] Koul, N. (2025, June 2). Getting started with Gradio Python library. Medium.<https://medium.com/@nimritakoul01/getting-started-with-gradio-python-library-49e59e363c66>
- [19] Transfer learning and fine-tuning in NLP. GeeksforGeeks. (n.d.).<https://www.geeksforgeeks.org/nlp/transfer-learning-and-fine-tuning-in-nlp/>
- [20] Fine-tuning using LoRA and QLoRA. GeeksforGeeks. (n.d.).<https://www.geeksforgeeks.org/deep-learning/fine-tuning-using-lora-and-qlora/>

[21] Tayal, R. (2024, May 15). My experiences with finetuning LLMs using LoRA. Medium.<https://medium.com/@rachittayal7/my-experiences-with-finetuning-llms-using-lora-b9c90f1839c6>

6 APPENDIX

Figure 2: Training&Validation Loss Curves

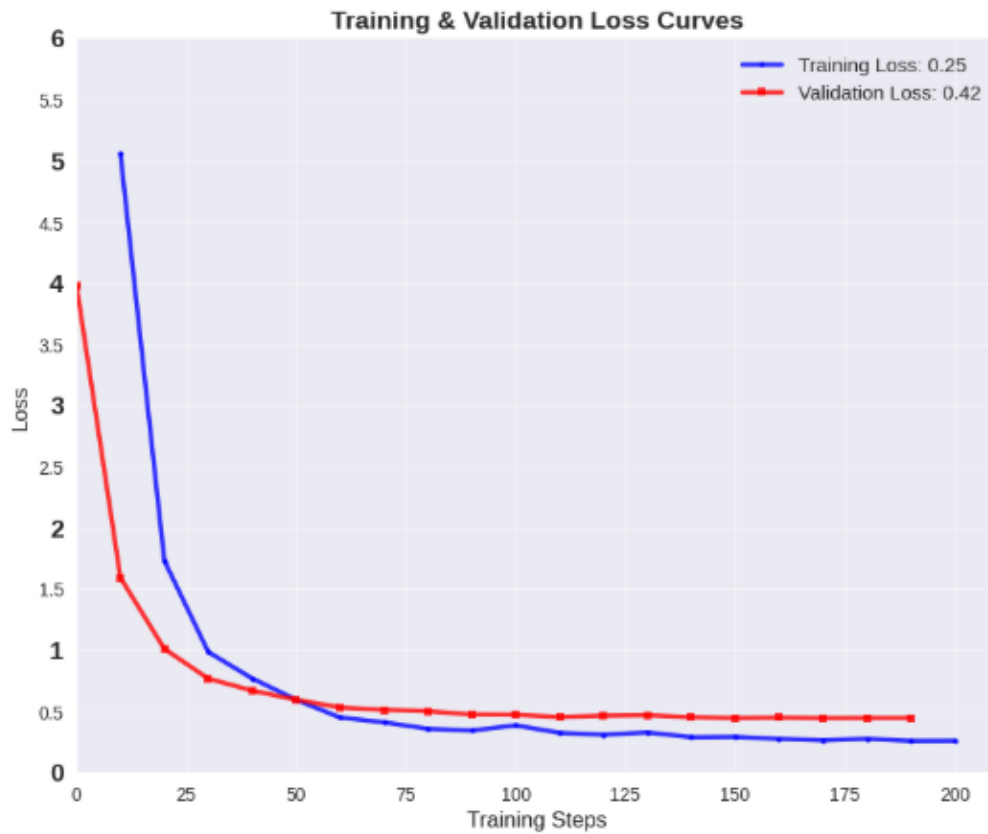


Figure 3: Response Time Distribution

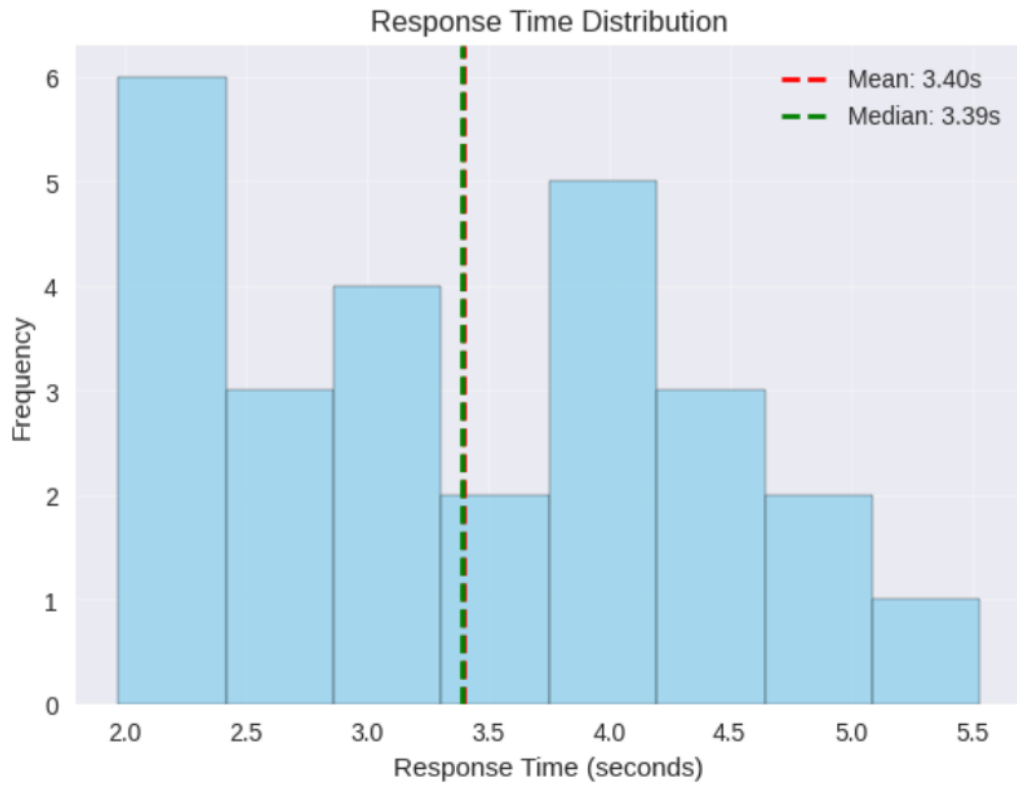


Figure 4: Response Length per Question

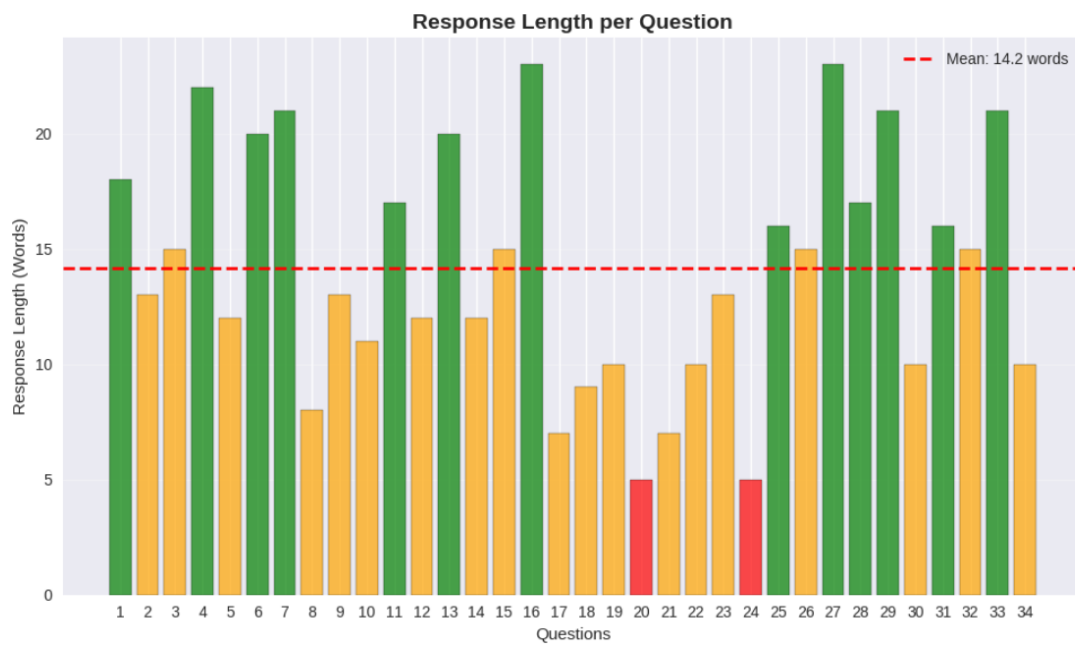


Figure 5: RAG System Performance Metrics

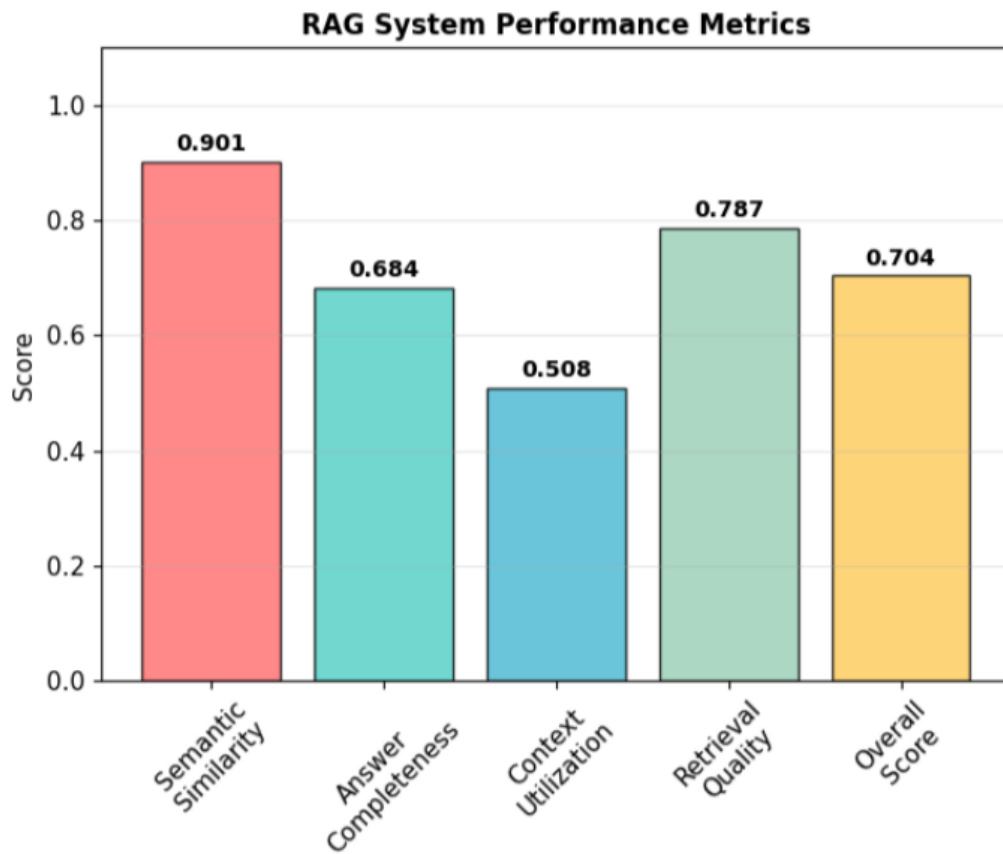


Figure 6: Context Usage Across Questions

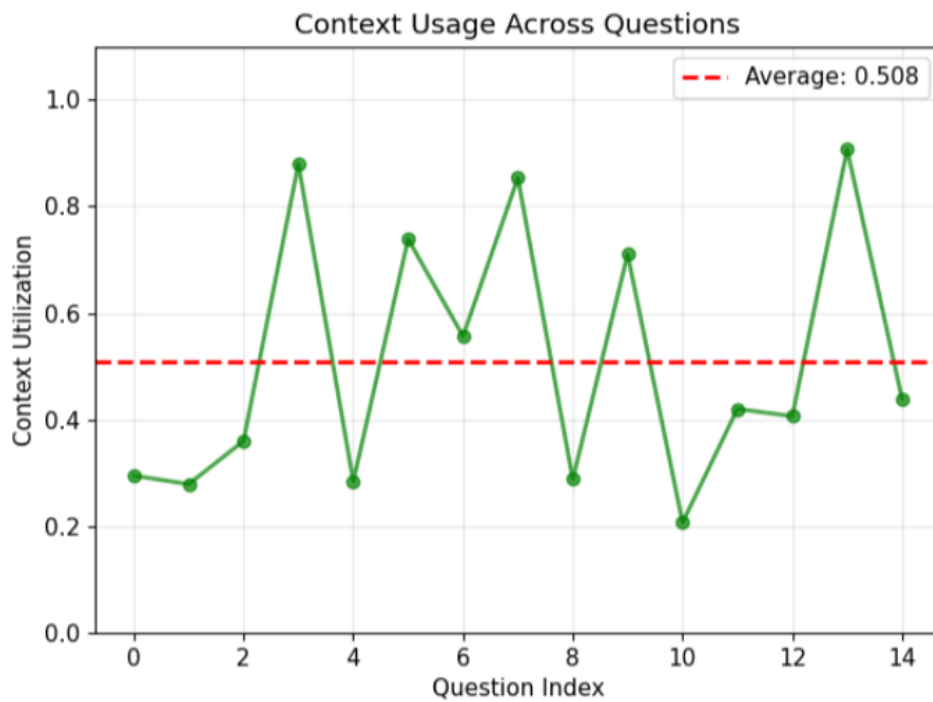


Figure 7: Correlation Matrix for RAG

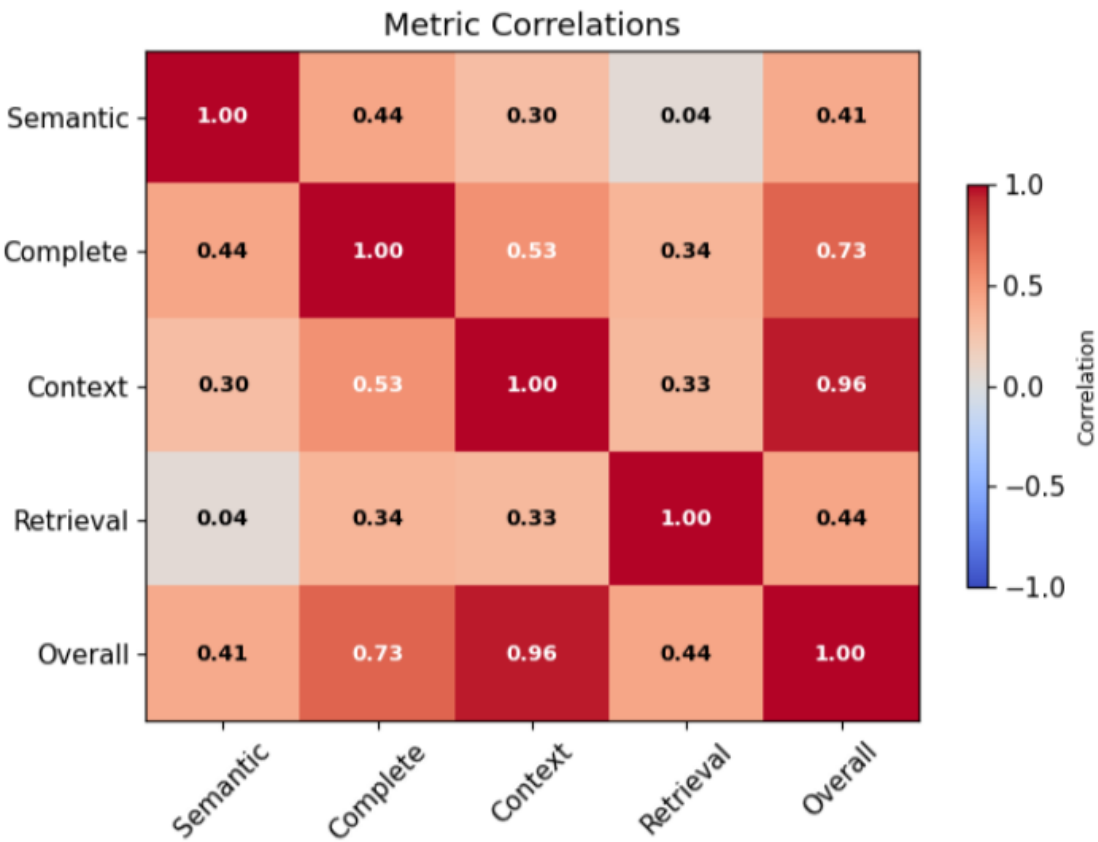


Figure 8: Interface - With a Question I Asked as an Example

**Gemma 3 + RAG Soru-Cevap Sistemi**
Fine-tuned Gemma 3 modeli ile ChromaDB tabanlı RAG sistemi

Soru Sorma Bölümü

Sorunuzu buraya yazın

explain me the difference between 8-bit quantization and 4-bit quantization

Soruyu Gönder

Temizle

Gelişmiş Ayarlar

Sistem Durumu

Anlık Durum

- Model: Yüklü ve hazır
- Device: cuda:0
- RAG System: 151 doküman yüklü
- Embedding Model: Unknown
- GPU Memory: 5.0GB allocated, 5.2GB cached

Durumu Yenile

Sistem Testi

Test Sonucu

Yanıt ve Detaylar

Model Yanıtı

8-bit quantization offers better accuracy with moderate memory savings, while 4-bit quantization saves more memory but may reduce accuracy slightly.

İşlem Bilgileri

Processing Time: 4.87s | **Mode:** RAG

Kaynak Bilgileri (RAG)

Örnek Sorular

- Machine learning nedir ve nasıl çalışır?...
- Fine-tuning ile transfer learning arasın...
- Embedding modelleri nasıl çalışır?...

Teknik Bilgiler: Gemma 3 4B + Unsloth + ChromaDB + E5 Embeddings

Powered by: Gradio Web Interface

Environment: Google Colab

16